

Guatemala, 20/07/2026

Universidad del Valle de Guatemala

Departamento de Ciencias de la Computación

Ingeniería en Ciencias de la Computación y Tecnologías de la Información

Data Science, sección 30



Laboratorio 3

Deep Learning

Belén Monterroso 231497

Melisa Mendizabal 23778

Renato Rojas 23813

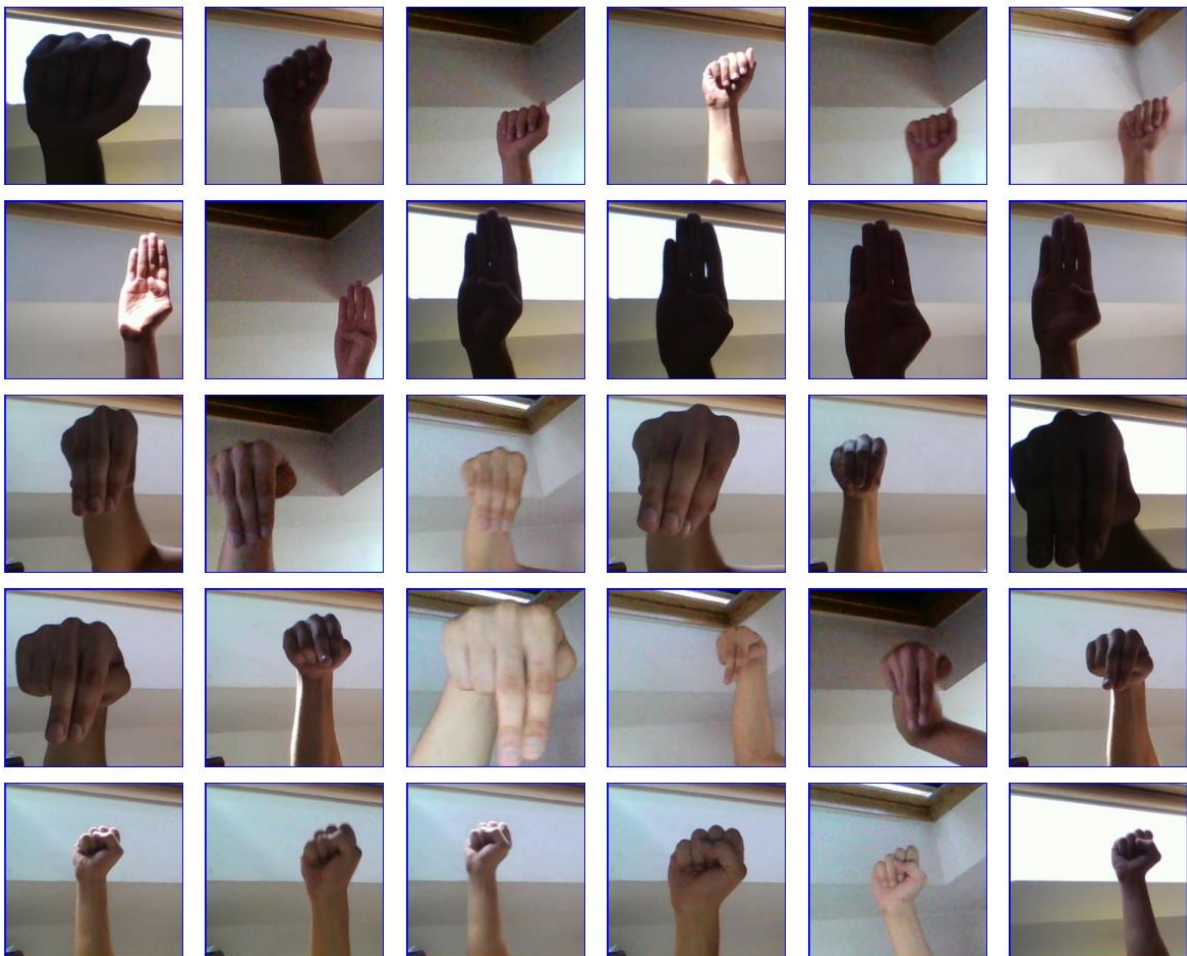
Introducción

El reconocimiento del lenguaje de señas es una aplicación de la inteligencia artificial que puede contribuir al desarrollo de herramientas de comunicación más accesibles. En este laboratorio se trabaja con un conjunto de imágenes de señas del alfabeto ASL, aplicando técnicas de análisis exploratorio y preprocesamiento para preparar los datos. Posteriormente, se implementan diferentes modelos de aprendizaje automático, principalmente redes neuronales convolucionales (CNN), además de una red completamente conectada y un algoritmo tradicional, con el objetivo de comparar su desempeño en la clasificación de las señas. También se experimenta con técnicas de aumento de datos y se realizan pruebas con señas realizadas por los integrantes del equipo para observar cómo los modelos se comportan con imágenes diferentes a las del conjunto de entrenamiento.

Análisis exploratorio

1. Muestre ejemplos de al menos 5 letras distintas del alfabeto ASL, incluyendo varias muestras de una misma letra para observar la variabilidad dentro de una clase.

Variabilidad dentro de cada clase (mismas letras, distintas manos/condiciones)



En la imagen se representan cinco letras distintas del alfabeto de señas ASL organizadas por filas, mostrando seis capturas de cada una para observar su variabilidad interna. Las letras seleccionadas fueron 'A', 'B', 'M', 'N,' y 'S'. A pesar de que las imágenes de una misma fila corresponden exactamente a la misma letra, se aprecian diferencias significativas en el entorno y la toma. Existen variaciones drásticas en la iluminación (desde manos casi en silueta por el contraluz de la ventana hasta tonos más oscuros y sombreados), cambios en la distancia y tamaño de la mano dentro del encuadre, e inclinaciones o ángulos ligeramente distintos al realizar el gesto. Esta diversidad refleja las condiciones reales a las que se enfrenta el modelo, demostrando que una misma clase debe ser reconocida con precisión sin importar quién haga la seña, la iluminación de la habitación o la posición de la cámara.

2. Haga un análisis exploratorio de los datos para entenderlos mejor y documente todos los hallazgos. Especifique la resolución y formato de las imágenes, la distribución de clases (¿está balanceado el dataset?), qué letras se confunden visualmente entre sí (por ejemplo M/N/S o U/V/R), y describa cómo definió su propio conjunto de entrenamiento/validación/prueba a partir de los datos disponibles.

- **Cantidad de Imágenes:**

Este dataset contiene un total de 87,000 imágenes. Estas se encuentran divididas en varias carpetas dependiendo de la letra o carácter que representan

- **Cantidad de clases:**

El dataset contiene un total de 29 clases diferentes. Dentro de estas se encuentran las letras de la A hasta la Z, no incluye la letra Ñ debido a que se basa en el lenguaje de señas en inglés. También se encuentran clases como 'del', que es utilizada para descartar el último símbolo mostrado; 'space', para representar los espacios entre caracteres; y la clase 'nothing', para que los modelos puedan tener una referencia sobre el fondo vacío y la ausencia de símbolos. Es importante resaltar que estas señas solo representan caracteres para ser utilizados en el deletreo, mas no señas más complejas que representen palabras completas.

- **Distribución de imágenes por clase:**

Como se puede observar la distribución de clases es homogénea. Esto quiere decir que las 29 clases encontradas contienen un total de 3,000 imágenes cada una. Esto representa un data set balanceado perfecto para el entrenamiento de datos sin necesidad de ajuste previo.



- **Analizar:**

- **Resolución:**

Todas las imágenes poseen una resolución de 200 x 200 píxeles, tal como lo indica la fuente de donde fue tomado el data set. Esta resolución es idónea para ser utilizada por modelos, ya que una resolución muy alta puede consumir muchos recursos de cómputo.

- **Formato:**

Todas las imágenes poseen el mismo formato de imagen (.jpg). Este análisis es importante ya que identifica de antemano imágenes dañadas, archivos de 0 bytes o formatos no deseados (como .gif o .bmp) que detendrían el pipeline de datos a mitad de un entrenamiento largo.

- **Modos de color:**

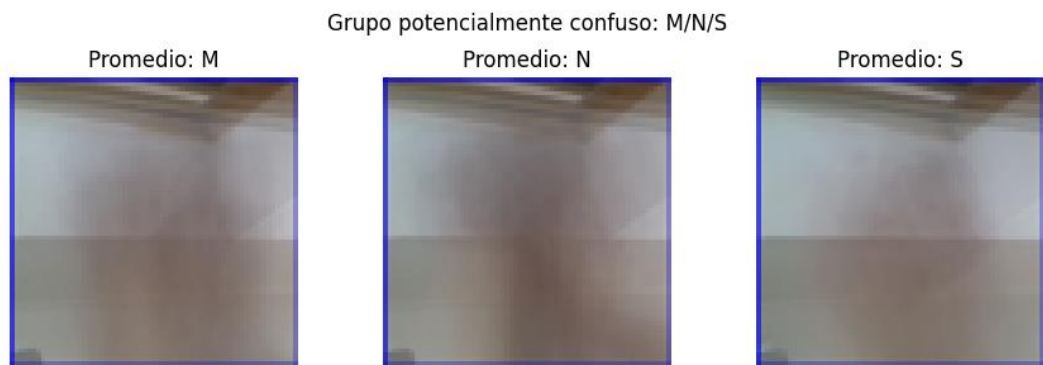
Todas las imágenes poseen el mismo modo de color, RGB sin identificador de transparencia.

- **Identificar letras visualmente similares:**

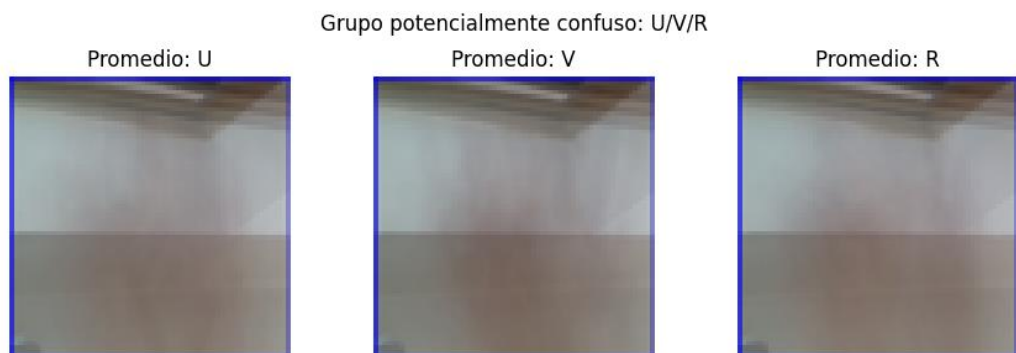
Este análisis es fundamental porque permite medir de forma cuantitativa qué tan difíciles de diferenciar son las letras antes de entrenar el modelo, anticipando sus posibles puntos ciegos. Al calcular la distancia matemática entre las imágenes promedio de cada clase, se transforma una observación visual en datos concretos, identifica exactamente qué señas comparten un patrón tan similar que corren el riesgo de confundirse. Esta información permite justificar preventivamente los futuros errores de clasificación, evaluar si se requiere mayor resolución en las imágenes y definir técnicas de procesamiento focalizadas para los pares más críticos.

Ejemplos solicitados:

- M/N/S



- U/V/R



Adicional a esto, se realizó una comparación entre todas las clases. Para este análisis se calculó una imagen promedio por clase, se tomó una muestra de imágenes de cada letra, se redujeron a una resolución fija (64×64) y se promediaron píxel a píxel, obteniendo así una "huella visual" representativa de cada seña. Estas 29 imágenes promedio se aplanaron en vectores numéricos y se compararon entre sí calculando la distancia euclidiana de todos los pares posibles de forma vectorizada (con `pdist/squareform` de `scipy`). Una distancia pequeña entre dos clases indica que sus señas son visualmente muy parecidas en promedio (mayor riesgo de que el modelo las confunda), mientras que una distancia grande indica letras claramente distinguibles, lo que permite identificar automáticamente, sin necesidad de inspección manual exhaustiva, qué pares de letras son más propensos a confusión.

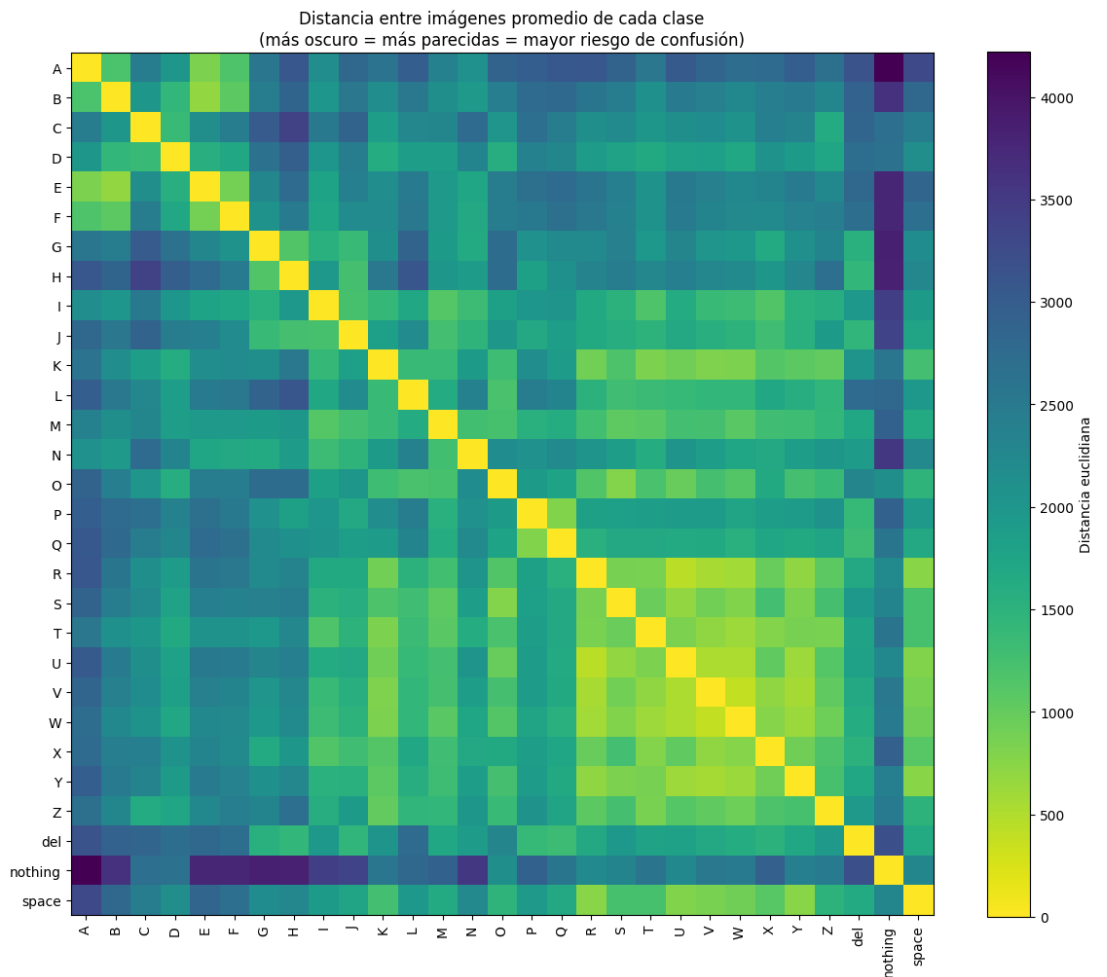
Los 10 pares de letras MÁS parecidas (mayor riesgo de confusión para el modelo):

Clase 1	Clase 2	Distancia
V	W	388.22
R	U	441.18
U	W	521.46
U	V	525.01
R	V	546.10
V	Y	568.14
R	W	592.57

U	Y	616.95
T	W	619.61
W	Y	630.61

Los 5 pares MÁS distintos (referencia de qué tan grande es una diferencia 'normal')

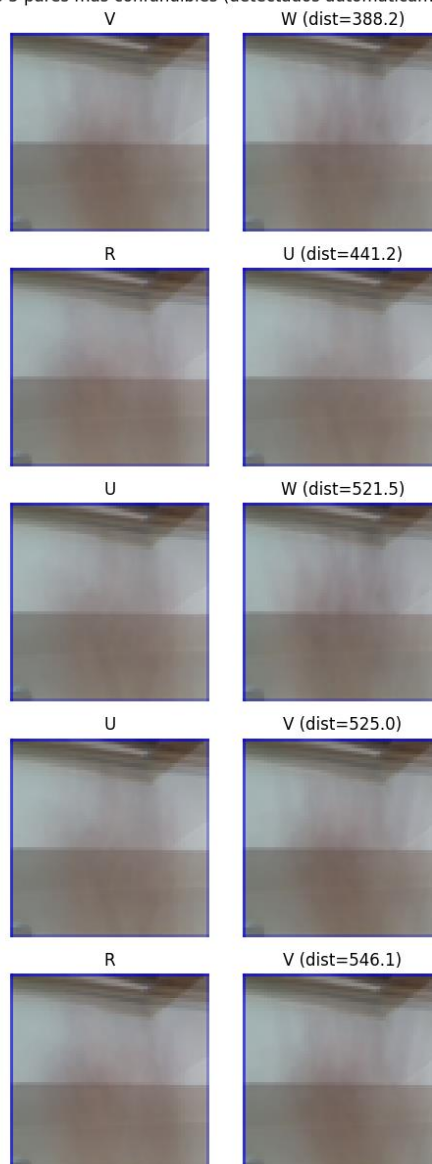
Clase 1	Clase 2	Distancia
F	nothing	3767.21
E	nothing	3771.80
H	nothing	3836.43
G	nothing	3856.59
A	nothing	4221.87



El mapa de calor revela una concentración de conflicto en el bloque formado por las letras R, U, V, W e Y, representadas por los tonos amarillos de menor distancia euclidiana. El bloque de mayor riesgo (R / U / V / W / Y): El par con mayor probabilidad de confusión en todo el dataset es V vs W con la distancia más baja registrado (388.22), seguido por R vs U (441.18) y U vs V (525.02). Esto ocurre

porque todas estas señas comparten una estructura idéntica basada en dos o tres dedos extendidos hacia arriba, haciendo que sus promedios visuales sean casi indistinguibles a nivel de píxeles.

Top 5 pares más confundibles (detectados automáticamente)



Comportamiento del grupo M / N / S: Aunque conceptualmente las letras M, N y S suelen considerarse parecidas por tratarse de puños cerrados, los datos muestran distancias considerablemente mayores entre sí (entre 1048.20 y 1874.95). Esto demuestra que el bloque U/V/R/W representa un desafío visual mucho más severo para el algoritmo que el grupo M/N/S.

Línea de base de separación: Las distancias entre las letras y la clase de control "nothing" superan valores de 3700 a 4200 (por ejemplo, A vs nothing con 4221.87),

lo que confirma que el modelo no tendrá problemas en distinguir la presencia de una mano frente a la ausencia de gestos.

- **Imágenes duplicadas:**

Clase	Cantidad
space	599
I	599
K	599
M	599
B	599
U	599
O	599
Y	599
E	599
W	599
C	600
D	600
nothing	600
F	600
A	600
del	600
J	600
Z	600
L	600
N	600
X	600
P	600
Q	600
R	600
S	600
T	600
V	600
H	600
G	600

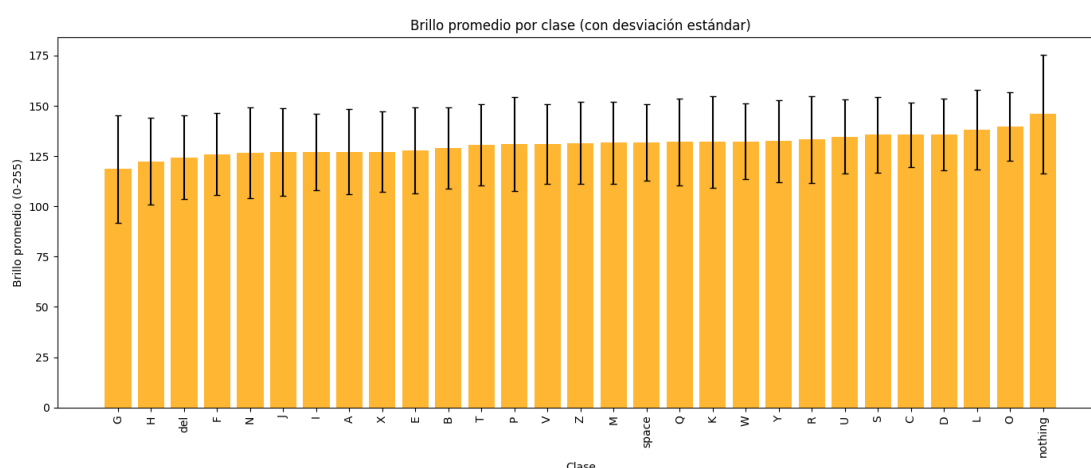
Para garantizar la calidad e integridad del data set, se implementó un flujo de limpieza de duplicados en dos capas: primero se generó una huella digital perceptualmente detallada (*pHash* con `hash_size=16`) para identificar posibles coincidencias y, posteriormente, se aplicó una verificación píxel a píxel midiendo el Error Cuadrático Medio ($MSE < 15$) para descartar falsos positivos. De una muestra inicial de 17,400 imágenes, el análisis por *hash* detectó 22 candidatas a duplicadas, confirmando una colisión nula entre clases distintas (0 grupos *cross-*

clase), lo que garantiza la ausencia de contaminación de etiquetas. La validación rigurosa por MSE confirmó únicamente 9 duplicados reales, reduciendo el dataset a 17,391 imágenes sin alterar la representatividad de la muestra, la cual mantuvo una distribución prácticamente perfecta y balanceada de entre 599 y 600 imágenes por cada una de las 29 clases.

- **Brillo y varianza de la muestra:**

Clase	Mean	Std	Diferencia absoluta
nothing	145.913743	29.451503	14.943885
G	118.556123	26.654049	12.413735
O	139.705852	16.863233	8.735994
H	122.321211	21.608931	8.648648
L	138.189337	19.810599	7.219478

Se calculó el nivel de brillo promedio de cada imagen (en una escala de 0 a 255) y se comparó el comportamiento de la iluminación entre las distintas clases mediante su valor medio y su desviación estándar, identificando las letras que más se alejan del promedio general. Permite verificar que el modelo de inteligencia artificial no aprenda "sesgos de iluminación". Si una letra específica fuera consistentemente más clara o más oscura que las demás, el algoritmo podría cometer el error de memorizar qué tan iluminada está la foto para clasificarla, en lugar de aprender la forma real del gesto de la mano.



La gran mayoría de las letras mantienen un brillo uniforme y controlado, ubicándose en un rango de entre 120 y 135 puntos.

La clase nothing (sin mano) es la más brillante con un promedio de 145.91, lo cual es lógico ya que no hay un cuerpo que proyecte sombra sobre el fondo. Por el

contrario, la letra G resultó ser la más oscura con un promedio de 118.56. La desviación estándar observada en todas las clases (las barras verticales del gráfico) confirma que el dataset cuenta con variaciones de luz realistas dentro de cada categoría, lo que ayudará a que el modelo sea más robusto ante diferentes entornos de iluminación.

- **Definición de su propio conjunto de entrenamiento/validación/prueba:**

La división del conjunto de datos se definió en una proporción 70% para entrenamiento, 15% para validación y 15% para prueba (12,173 / 2,609 / 2,609 imágenes respectivamente), un esquema estándar en proyectos de deep learning que asigna la mayor parte de los datos al aprendizaje del modelo, dejando al mismo tiempo conjuntos suficientemente grandes para ajustar hiperparámetros (validación) y para evaluar el desempeño final de forma independiente (prueba). La división se realizó de manera estratificada por clase, de modo que cada uno de los tres conjuntos mantiene la misma proporción de las 29 clases presentes en la submuestra original, evitando que alguna letra quedara subrepresentada o ausente en train, validación o test. Además, esta división se hizo después de eliminar los duplicados detectados, asegurando que ninguna imagen (o una casi idéntica) apareciera simultáneamente en más de un conjunto, lo cual habría inflado artificialmente las métricas de evaluación al permitir que el modelo "viera" durante el entrenamiento imágenes prácticamente iguales a las usadas para medir su desempeño.

Preprocesamiento de las imágenes

3. Documente el preprocesamiento que le hizo a las imágenes antes de entrenar (normalización, cambios de resolución, filtros, etc.)

Para el preprocesamiento del dataset se partió de las 87,000 imágenes originales, con un tamaño de 200 por 200 píxeles, en formato JPEG y modo RGB. De este conjunto se tomó una submuestra estratificada de 600 imágenes por clase, dentro del rango de 500 a 800 imágenes recomendado en el enunciado. Esta cantidad permitió mantener una buena diversidad de datos y, al mismo tiempo, reducir el tiempo de cómputo necesario para entrenar los diferentes modelos durante el laboratorio.

Antes de realizar la división de los datos, se llevó a cabo una limpieza de imágenes duplicadas mediante un hash perceptual, utilizando un tamaño de hash de 16 para reducir la posibilidad de considerar como duplicadas imágenes que únicamente compartieran fondos similares. Cada coincidencia encontrada fue comprobada adicionalmente mediante una comparación de diferencia de píxeles. De esta manera, se evitó que imágenes prácticamente idénticas quedaran distribuidas entre diferentes

conjuntos y pudieran generar una fuga de información durante el entrenamiento. Esta limpieza se realizó antes de dividir los datos, con el objetivo de asegurar que los conjuntos de entrenamiento, validación y prueba contuvieran imágenes independientes entre sí.

Finalmente, a cada conjunto se le aplicó una reducción de resolución de 200 por 200 a 64 por 64 píxeles. Esta reducción permitió disminuir la carga computacional sin perder la información visual relevante de las señas. También se normalizaron los valores de los píxeles al rango de 0 a 1, dividiendo cada valor entre 255, lo cual facilita el proceso de entrenamiento de las redes neuronales. Por último, las etiquetas de las clases se transformaron a valores numéricos mediante un LabelEncoder, ajustado únicamente utilizando el conjunto de entrenamiento. Esto permitió evitar que la información de los conjuntos de validación y prueba influyera en el proceso de codificación.

Modelos de Deep Learning para imágenes

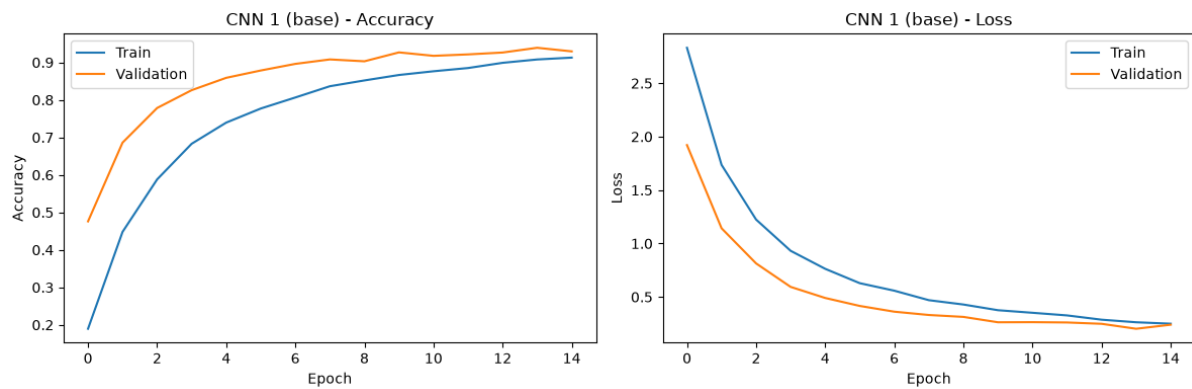
4. Haga al menos 2 modelos de Deep Learning (redes neuronales convolucionales), determine la efectividad de cada uno y seleccione el mejor.

Selección de modelos:

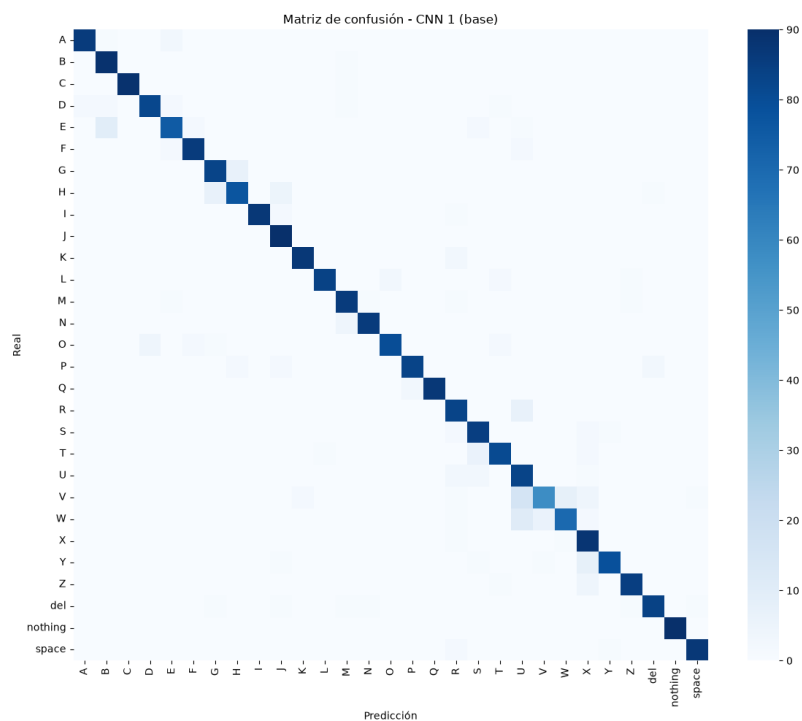
- Red Neuronal: Utilizaremos la red neuronal fully-connected como modelo base para comparar el desempeño de las arquitecturas. Se espera que este sea el peor modelo, pues no tiene la arquitectura adecuada para tratar con este tipo de datos (imágenes).
- CNN1: Esta permite extraer de forma automática características espaciales, lo hace con capas de convolucionales. Se tendrá como una configuración bastante default.
- CNN2: Para evaluar si un número mayor de capas logra mejorar el reconocimiento. Básicamente, una mejoría de profundidad de capas y mayor variabilidad de hiperparámetros respecto al CNN1 (se agregará un pequeño grid search).

MODELOS

CNN #1:



- Accuracy: obtuvo un accuracy de 0.9295 en validation, y de 0.9237 en test.
- Loss: tuvo un loss de 0.2402 en validation, y de 0.2561 en test.
- Matriz de confusión



- Tiempo de entrenamiento: tardó 105.2 segundos (aprox. 2min).
- Resultados de validation/test
Val accuracy: 0.9295 | Val loss: 0.2402
Test accuracy: 0.9237 | Test loss: 0.2561

Estas fueron sus capas:

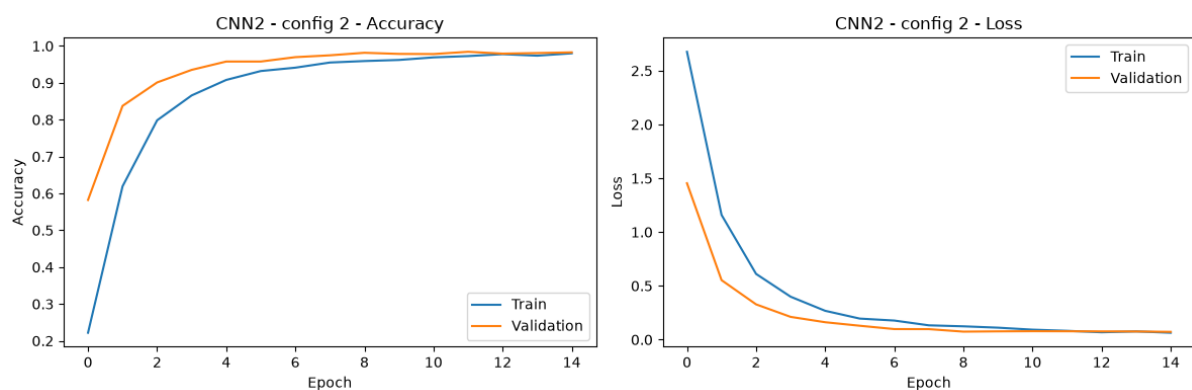
Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 64, 64, 32)	896
max_pooling2d (MaxPooling2D)	(None, 32, 32, 32)	0
conv2d_1 (Conv2D)	(None, 32, 32, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 16, 16, 64)	0
flatten_1 (Flatten)	(None, 16384)	0
dense_3 (Dense)	(None, 128)	2,097,280
dropout_2 (Dropout)	(None, 128)	0
dense_4 (Dense)	(None, 29)	3,741

Usando ReLU para las conv2d, y la “dense_3”. Usando softmax para la “dense_4” y LR adam = 0.001.

CNN #2

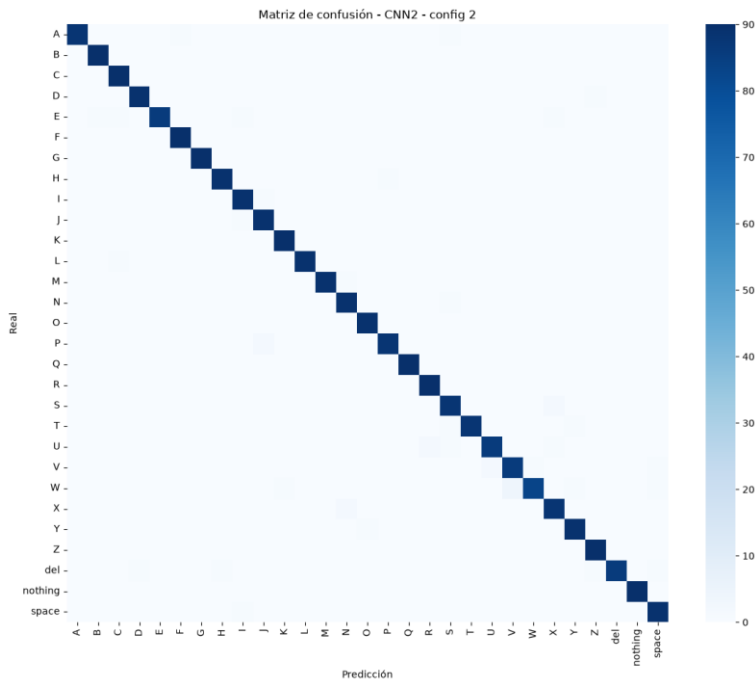
Se utilizó un pequeño grid search para los valores de esta segunda CNN; específicamente, solo se midieron 2 distintas CNN, y entre ellas cuál era la mejor, ambas con valores distintos a CNN1 recién presentado. De estas dos, la que mejor desempeño tuvo fue aquella con los siguientes hiperparámetros: "dropout": 0.5, "learning_rate": 0.001, "batch_size": 64, "epochs": 15.

Aquí se presentarán únicamente los resultados de esta CNN con mejor desempeño; para ver la otra, consultar en el archivo “lab3_cnn.ipynb” del repositorio.



- Accuracy: obtuvo un accuracy de 0.9828 en validation, y de 0.9839 en test.
- Loss: tuvo un loss de 0.0724 en validation, y de 0.0554 en test.

- Matriz de confusión



- Tiempo de entrenamiento: tardó 338.9 segundos (aprox. 6min).
- Resultados de validation/test
Val accuracy: 0.9828 | Val loss: 0.0724
Test accuracy: 0.9839 | Test loss: 0.0554

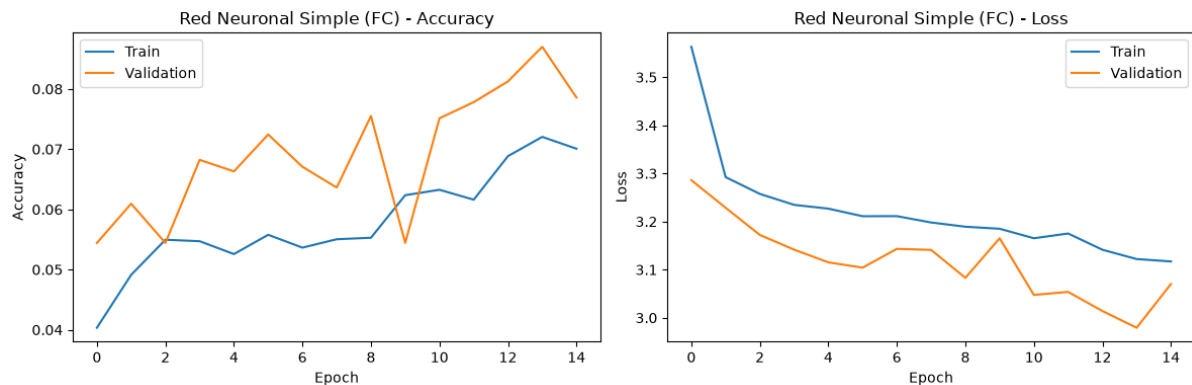
Estas fueron sus capas:

Layer (type)	Output Shape	Param #
conv2d_7 (Conv2D)	(None, 64, 64, 32)	896
conv2d_8 (Conv2D)	(None, 64, 64, 32)	9,248
max_pooling2d_5 (MaxPooling2D)	(None, 32, 32, 32)	0
conv2d_9 (Conv2D)	(None, 32, 32, 64)	18,496
conv2d_10 (Conv2D)	(None, 32, 32, 64)	36,928
max_pooling2d_6 (MaxPooling2D)	(None, 16, 16, 64)	0
conv2d_11 (Conv2D)	(None, 16, 16, 128)	73,856
max_pooling2d_7 (MaxPooling2D)	(None, 8, 8, 128)	0
flatten_3 (Flatten)	(None, 8192)	0
dense_7 (Dense)	(None, 256)	2,097,408
dropout_4 (Dropout)	(None, 256)	0
dense_8 (Dense)	(None, 29)	7,453

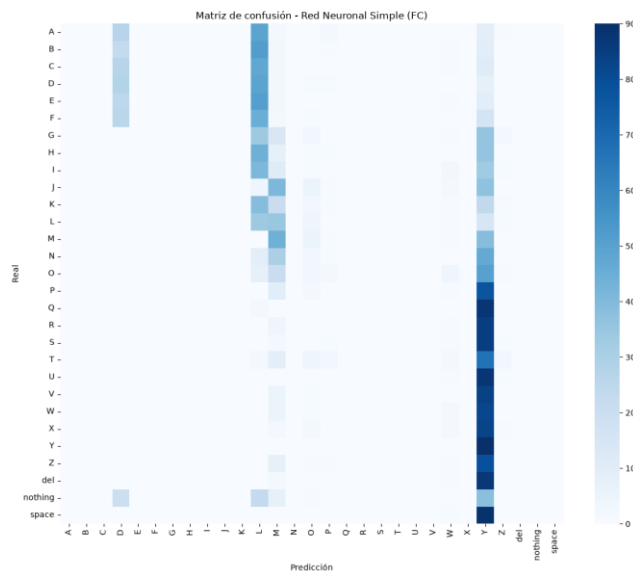
Usando ReLU para todas, exceptuando la última densa, siendo esa softmax. Siendo el LR adam, pero variable según los 2 CNN propuestos explicados (grid search).

5. Haga un modelo de redes neuronales simple (fully-connected), determine su efectividad y compárelo contra sus modelos de CNN:

Como se esperaba, este tuvo el peor desempeño entre todos los modelos, teniendo un rendimiento realmente malo.



- Accuracy: obtuvo un accuracy de 0.0789 en validation, y de 0.0770 en test.
- Loss: tuvo un loss de 3.0701 en validation, y de 3.0783 en test.
- Matriz de confusión



- Tiempo de entrenamiento: tardó 117.5 segundos (aprox. 2min).
- Resultados de validation/test
Val accuracy: 0.0786 | Val loss: 3.0701
Test accuracy: 0.0770 | Test loss: 3.0783

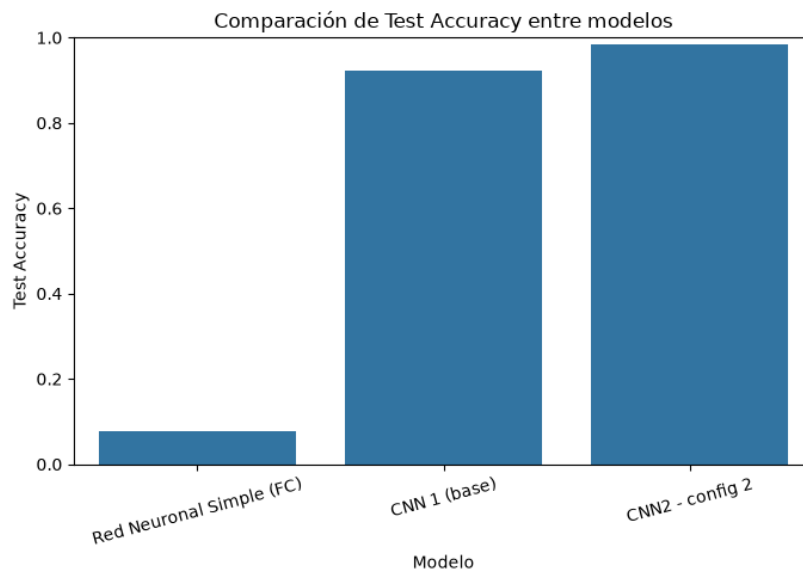
Para este modelo de FC, estas fueron sus parámetros y capas:

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 12288)	0
dense (Dense)	(None, 512)	6,291,968
dropout (Dropout)	(None, 512)	0
dense_1 (Dense)	(None, 256)	131,328
dropout_1 (Dropout)	(None, 256)	0
dense_2 (Dense)	(None, 29)	7,453

ReLU para densa 1 y 2. Softmax para densa 3. Learning rate de adam de 0.001. El valor del dropout de 0.3 para ambos casos.

Comparación:

Los 3 modelos tuvieron 15 épocas para entrenar. Y su resultado comparado se ve así:



Como se ve, existe una gran mejora por parte de los modelos convolucionales respecto a la red neuronal plana FC. Esto es por la naturaleza de los datos (imágenes), pues la plana trata a cada fragmento de la imagen como una neurona independiente, mientras que las convolucionales, gracias a su factor del kernel deslizando, es que tiene noción sobre la vecinidad de los píxeles, así como la no-independencia de los mismos.

Se ve también que la segunda CNN fue la que tuvo mejor desempeño; por tanto, se podría concluir que, al tener más capas, se tiene una mejor predicción para este tipo de datos; aunque puede haber ocurrido que, al tener una mayor variabilidad de

hiperparámetros, también haya encontrado una mejor solución, sin tener todo el peso de la mejora el que existan más capas ocultas.

Si se tuviera que crear un sistema para producción, se recomendaría encarecidamente el uso de una CNN con una buena cantidad de capas, tal como ocurrió en la CNN2, respaldado por su elevado accuracy.

Modelo con otro algoritmo

6. Haga un modelo con cualquier otro algoritmo que el grupo seleccione (por ejemplo Random Forest, SVM o KNN), determine su efectividad y explique la base de su selección.

Cosas por hacer con la elección

- Preparar los datos para el algoritmo.
- Entrenar varios parámetros/configuraciones.
- Evaluar resultados.

	Modelo	C	Accuracy Train	Accuracy Validation	Tiempo (s)
0	SVM C=0.1	0.10	1.0000	0.612495	1806.950245
1	SVM C=0.01	0.01	0.9262	0.582215	957.979855

```
Mejor configuración:  
Modelo: SVM C=0.1  
C: 0.1
```

```
Mejor configuración: SVM C=0.1  
Accuracy Test: 0.6198  
Tiempo de predicción: 0.57 segundos
```

El mejor resultado del modelo SVM fue con $C = 0.01$ con un accuracy de 61.98% en prueba. El modelo obtuvo un accuracy del 100% en train, esto muestra un sobreajuste grande. El train también presentó un costo computacional grande con 1807 segundos. En comparación, $C=0.01$ redujo ligeramente el sobreajuste, pero obtuvo una menor accuracy de validación de 58.22%.

- Justificar por qué se escogió el algoritmo: Seleccionamos SVM como el modelo alternativo porque permite tener fronteras de decisión entre varias clases a

partir de las características de entrada. Para adaptar las imágenes, se transformaron de matrices a vectores de características. Entonces SVM puede trabajar directamente con la información de cada imagen sin utilizar capas convolucionales.

7. Hágale transformaciones a los datos como image augmentation y vuelva a entrenar sus modelos. Discuta los resultados y responda: ¿por qué un flip horizontal podría cambiar el significado de una señal en vez de solo aumentar los datos? ¿qué transformaciones sí tienen sentido para señales y cuáles no?

- Rotación.
- Zoom.
- Traslación.
- Cambios moderados de iluminación.
- Otros cambios que tengan sentido para las señales.

```
Epoch 1/15
381/381 ————— 28s 65ms/step - accuracy: 0.1240 - loss: 3.0507 - val_accuracy: 0.4048 - val_loss: 2.0439
Epoch 2/15
381/381 ————— 23s 60ms/step - accuracy: 0.4529 - loss: 1.7764 - val_accuracy: 0.7010 - val_loss: 0.9834
Epoch 3/15
381/381 ————— 24s 63ms/step - accuracy: 0.6514 - loss: 1.0624 - val_accuracy: 0.8605 - val_loss: 0.4717
Epoch 4/15
381/381 ————— 22s 59ms/step - accuracy: 0.7549 - loss: 0.7343 - val_accuracy: 0.9088 - val_loss: 0.2991
Epoch 5/15
381/381 ————— 23s 61ms/step - accuracy: 0.8226 - loss: 0.5360 - val_accuracy: 0.9345 - val_loss: 0.2025
Epoch 6/15
381/381 ————— 23s 60ms/step - accuracy: 0.8576 - loss: 0.4289 - val_accuracy: 0.9590 - val_loss: 0.1612
Epoch 7/15
381/381 ————— 22s 58ms/step - accuracy: 0.8760 - loss: 0.3611 - val_accuracy: 0.9498 - val_loss: 0.1515
Epoch 8/15
381/381 ————— 22s 58ms/step - accuracy: 0.8990 - loss: 0.3009 - val_accuracy: 0.9490 - val_loss: 0.1407
Epoch 9/15
381/381 ————— 22s 56ms/step - accuracy: 0.9058 - loss: 0.2786 - val_accuracy: 0.9739 - val_loss: 0.0787
Epoch 10/15
381/381 ————— 21s 56ms/step - accuracy: 0.9212 - loss: 0.2309 - val_accuracy: 0.9770 - val_loss: 0.0726
Epoch 11/15
381/381 ————— 22s 56ms/step - accuracy: 0.9330 - loss: 0.2048 - val_accuracy: 0.9793 - val_loss: 0.0662
Epoch 12/15
381/381 ————— 22s 58ms/step - accuracy: 0.9320 - loss: 0.1996 - val_accuracy: 0.9774 - val_loss: 0.0712
Epoch 13/15
...
```

```

Epoch 3/15
381/381 ————— 24s 63ms/step - accuracy: 0.6514 - loss: 1.0624 - val_accuracy: 0.8605 - val_loss: 0.4717
Epoch 4/15
381/381 ————— 22s 59ms/step - accuracy: 0.7549 - loss: 0.7343 - val_accuracy: 0.9088 - val_loss: 0.2991
Epoch 5/15
381/381 ————— 23s 61ms/step - accuracy: 0.8226 - loss: 0.5360 - val_accuracy: 0.9345 - val_loss: 0.2025
Epoch 6/15
381/381 ————— 23s 60ms/step - accuracy: 0.8576 - loss: 0.4289 - val_accuracy: 0.9590 - val_loss: 0.1612
Epoch 7/15
381/381 ————— 22s 58ms/step - accuracy: 0.8760 - loss: 0.3611 - val_accuracy: 0.9498 - val_loss: 0.1515
Epoch 8/15
381/381 ————— 22s 58ms/step - accuracy: 0.8990 - loss: 0.3009 - val_accuracy: 0.9490 - val_loss: 0.1407
Epoch 9/15
381/381 ————— 22s 56ms/step - accuracy: 0.9058 - loss: 0.2786 - val_accuracy: 0.9739 - val_loss: 0.0787
Epoch 10/15
381/381 ————— 21s 56ms/step - accuracy: 0.9212 - loss: 0.2309 - val_accuracy: 0.9770 - val_loss: 0.0726
Epoch 11/15
381/381 ————— 22s 56ms/step - accuracy: 0.9330 - loss: 0.2048 - val_accuracy: 0.9793 - val_loss: 0.0662
Epoch 12/15
381/381 ————— 22s 58ms/step - accuracy: 0.9320 - loss: 0.1996 - val_accuracy: 0.9774 - val_loss: 0.0712
Epoch 13/15
...
Accuracy train: 0.9900
Accuracy validation: 0.9805
Accuracy test: 0.9778
Tiempo: 354.27 segundos

```

Para mejorar la capacidad de generalización del modelo usamos la augmentation sobre las imágenes de entrenamiento. Lo volvimos a entrenar con CNN2 utilizando las imágenes transformadas. Tuvimos una mejora progresiva de train como en test. En la primera parte, el modelo nos presentó un accuracy de 12.40% en entrenamiento y 40.48% para la validación. Mientras avanzaron las épocas, el desempeño aumentó.

Para la época 6 el accuracy fue de 95.90% y para la 9 fue de 97.39%. El modelo obtuvo un 99% de accuracy en train y 97.78 en test. El tiempo del entrenamiento fue de 354.27 segundos.

Dados los resultados de entrenamiento, validación y prueba podemos decir que el modelo presenta una buena capacidad de generalización. Esto en comparación con el SVM donde tiene una diferencia enorme entre train y test. Este modelo logró aprender patrones robustos pues en el train llegó a aproximadamente 98.85%.

Para este contexto es importante elegir bien las técnicas de augmentation para que las imágenes tengan el mismo significado.

Un horizontal flip en problemas comunes puede ser útil ya que un objeto normalmente mantiene su categoría después de ser reflejado. Pero en el lenguaje de señas, la orientación, posición y lateralidad de las manos es de importancia para identificar una señal. Al reflejar una mano derecha en horizontal, puede parecer que es la izquierda y cambia la relación de la señal. Por esto no se puede asumir que la imagen reflejada mantiene el mismo significado.

Por otro lado, la rotación pequeña o el zoom sí tienen sentido, porque son variaciones que se presentan naturalmente. Cuando una persona hace una señal puede variar la

iluminación, el zoom o una pequeña rotación. Entonces, se tienen que evitar transformaciones grandes que puedan afectar el significado de la seña.

8. Pruebe el mejor modelo con fotos de señas hechas a mano por los integrantes del grupo (mínimo 5 letras distintas por integrante). Discuta el desempeño del modelo y los resultados obtenidos.

Resumen general:

sum

count

integrante

Belen

2

5

Melisa

3

6

Renato

2

6

Accuracy total en fotos propias: 41.2%

	integrante	archivo	real	predicho	confianza	correcto
0	Renato	./TestPropio/Renato/R.jpg	R	R	0.9247	True
1	Renato	./TestPropio/Renato/E.jpg	E	S	0.8973	False
2	Renato	./TestPropio/Renato/N.jpg	N	N	0.9069	True
3	Renato	./TestPropio/Renato/A.jpg	A	T	0.7798	False
4	Renato	./TestPropio/Renato/T.jpg	T	Y	0.9721	False
5	Renato	./TestPropio/Renato/O.jpg	O	Y	0.7735	False
6	Melisa	./TestPropio/Melisa/M.jpg	M	W	1.0000	False
7	Melisa	./TestPropio/Melisa/E.jpg	E	nothing	0.9121	False
8	Melisa	./TestPropio/Melisa/L.jpg	L	L	0.9989	True
9	Melisa	./TestPropio/Melisa/I.jpg	I	I	0.9992	True
10	Melisa	./TestPropio/Melisa/S.jpg	S	S	1.0000	True
11	Melisa	./TestPropio/Melisa/A.jpg	A	Z	0.5396	False
12	Belen	./TestPropio/Belen/B.jpg	B	B	0.9993	True
13	Belen	./TestPropio/Belen/E_1.jpg	E	S	0.5371	False
14	Belen	./TestPropio/Belen/L.jpg	L	L	0.9137	True
15	Belen	./TestPropio/Belen/E_2.jpg	E	S	0.9975	False
16	Belen	./TestPropio/Belen/N.jpg	N	T	0.5334	False

A pesar de ser el mejor modelo en cuanto al conjunto de validation y test (con un abrumante 98% de accuracy en dichos conjuntos), al colocar nuestras propias fotos con el lenguaje de señas, cada uno recreando su propio nombre con sus manos, apenas si

se logró un accuracy de 41%. Esto se puede deber a factores como la iluminación de las fotos iniciales (tanto train como validation/test) es distinta a la nuestra, lo que altera el análisis de nuestras fotos. También puede influir el tono de piel distinto, el nuestro respecto al de la persona que creó el dataset, pues la red entrenó para comprender en totalidad sus manos; al colocar las nuestras, puede llegar a confundir algunas letras, no tanto por la posición de los dedos o forma de la mano, sino la interpretación del tono de piel. Otro aspecto que también pudo haber influido es el color del fondo; mientras que en el dataset original parece que la persona está con la mano al aire, nosotros decidimos colocarla frente a una pared (y de distintos colores), lo que puede generar también una peor interpretación de nuestras imágenes.

Básicamente, puede que, en la práctica real, este modelo haya experimentado overfitting respecto al individuo que creó el dataset original (su iluminación, fondo, tono de piel, etc.), no tanto a datos específicos de train/test. Por tanto, su uso comercial o científico al público abierto no sería recomendable tal como está; debería entrenarse mucho más con muchos más datos de una cantidad realmente amplia de personas, con muchos fondos y distintas iluminaciones.

9. Reflexione sobre accesibilidad y sesgo: ¿qué limitaciones tiene este dataset (tono de piel, ángulo de cámara, iluminación, tamaño de mano) para un producto real de accesibilidad? ¿Qué haría falta para llevar este prototipo a un caso de uso real como el de SignBridge?

Accesibilidad y sesgo

- Tono de piel.
- Iluminación.
- Ángulo de cámara.
- Tamaño de mano.
- Posición de la mano.
- Variabilidad entre personas.

El dataset en general tiene varias limitaciones, por ejemplo, una de las principales sería la falta de diversidad en cuanto a tonos de piel y también el ángulo desde el cual se toma la foto. El ángulo es importante porque pueden tener distintos tipos de iluminación que pueden cambiar el contraste del fondo con la mano, esto también hace que tenga una apariencia distinta. También el tono de piel hace que se reconozca mejor para unos mientras que para otros con poco contraste entre el fondo y el tono de piel cuesta más identificarlo.

Para poner mejorar el prototipo y llevarlo a un caso real sería indispensable poder tener un dataset más variado, con personas con distintas características físicas. También incluir fotos con ángulos más variados y extremos para que se pueda tener un mejor reconocimiento.

Repositorio: https://github.com/belennmm/DataS_Lab3_ASF

Documento: https://uvgt-my.sharepoint.com/:w:/g/personal/roj23813_uvg_edu_gt/IQDFb1AEFaaaRpMYAUsHy2mvAQGVwzQEXnq8X76lZ_q5BZY?e=JR6IT1